A Dataset Details

We evaluate STEER-KT on three knowledge tracing datasets: DBE-KT22 [1], NIPS34 [13], and XES3G5M [10]. We choose datasets with textual question or concept metadata (e.g., question content and concept description), which allows STEER-KT to utilize the semantic understanding capabilities of LLMs and better aligns with real-world educational scenarios.

The training set is used to estimate proxy question difficulties and retrieve peer-view evidence for STEER-KT. During inference, we use user-disjoint evaluation for DBE-KT22, as the dataset does not provide a predefined user-disjoint split. Specifically, we randomly select 20% of the users, resulting in 235 test users, while ensuring no overlap with the training users. For NIPS34, we use the entire provided test set, comprising 615 users. For XES3G5M, we randomly sample 500 users from the provided test set for evaluation. For each sampled user, we select one target interaction from the user's test sequence for evaluation. The same target users and interactions are used across all methods for fair comparison. Dataset statistics and label distributions for the test datasets are summarized in Table 4. Label distribution of the sampled test instances used for evaluation.

Table 4: Statistics of the datasets. Train/Test denotes the student and interaction split used in our experiments.

	DBE-KT22	NIPS34	XES3G5M
Statistics of the datasets			
# Questions	212	948	7,652
# Concepts	93	85	1,174
Avg. Seq. Length	134.3	259.1	304.5
# Train Students	948	4,918	14,453
# Test Students	235	615	500
# Train Interactions	128,895	1,382,661	4,400,806
# Test Interactions	29,981	51,184	152,361
Target Response Distribution			
Correct	137 (58.3%)	311 (50.6%)	405 (81.0%)
Incorrect	98 (41.7%)	304 (49.4%)	95 (19.0%)

B Implementation Details

Deep Learning-based KT Models. We use open-source code [4] to reproduce each model and adopt it in multi-concept setting according to [9]. We use the optimal hyperparameters recommended by the original paper for each model.

Training-Free LLM-based KT Models. In our implementation, we preserve multiple concepts associated with each question, allowing each method to operate under a multi-concept setting.

For HISE-KT, we follow the original paper by sampling $N = 100$ candidates for each meta-path. To reduce inference cost, we use GPT4.1-mini as the lightweight scoring model.

Experimental Setup. We report the mean and standard deviation over three independent runs with different random seeds for the DL-based KT methods and three repeated inference runs for the LLM-based KT methods. For a fair comparison, all LLM-based methods use GPT-5.2 as the underlying prediction model.

[4] https://github.com/pykt-team/pykt-toolkit

We implemented all models using PyTorch 2.9.1 with CUDA 12.8, and computed Macro-F1 and Micro-F1 using scikit-learn.

C Details of STEER-KT

Details of Self-Evidence Retrieval. To retrieve self-evidence, we compute the similarity between the target episode and prior candidate episodes from the target student's trajectory using three components: (1) trajectory similarity, (2) target-candidate difficulty similarity, and (3) temporal recency. Specifically, trajectory similarity measures the similarity between interaction sequences within two episodes, target-candidate difficulty measures the similarity between the difficulty of the target question and the candidate episode's target question, and temporal recency measures how close the candidate episode is to the target episode in time.

Trajectory Similarity Score. The trajectory similarity is defined in Section 3.3, aggregating the interaction-level similarity score according to their relative positions. Specifically, for two interactions $e_{u,i} = (q_{u,i}, r_{u,i})$ and $e_{u,j} = (q_{u,j}, r_{u,j})$, at the same relative position in the target and candidate episodes, respectively, we define their response-state similarity as

$$\begin{aligned} s(e_{u,i}, e_{u,j}) = \omega_r \mathbb{I}[r_{u,i} = r_{u,j}] \\ + \omega_d \left(1 - \left|d(q_{u,i}) - d(q_{u,j})\right|\right) \\ + \omega_\delta \left(1 - \frac{\left|\delta_{u,i} - \delta_{u,j}\right|}{2}\right), \end{aligned} \tag{1}$$

where

$$\delta_{u,i} = r_{u,i} - (1 - d(q_{u,i})) \text{ and } \delta_{u,j} = r_{u,j} - (1 - d(q_{u,j})). \tag{2}$$

where the three terms correspond to response consistency, question difficulty similarity, and performance similarity with respect to population-level expectations, respectively. We set $\omega_r = 0.30$, $\omega_d = 0.30$, and $\omega_\delta = 0.40$ in all experiments.

We further assign higher weights to more recent interactions within each episode:

$$S_{\text{traj}}(\mathcal{E}, \mathcal{E}') = \frac{\sum_{k=1}^{w} \alpha_k s(\phi(\mathcal{E}[k]), \phi(\mathcal{E}'[k]))}{\sum_{k=1}^{w} \alpha_k}, \tag{3}$$

where α_k assigns greater importance to more recent interactions within an episode. $\mathcal{E}[k]$ denotes the k-th interaction in episode $\mathcal{E}$. In our experiments, we set $w = 5$ and use position weights as $\alpha_k = [1.0, 1.0, 1.2, 1.5, 2.0]$.

Target-Difficulty Similarity Score. For a target question $q_{u,|\mathbf{s}_u|+1}$ and a candidate question $q_{u,\tau}$ following a candidate episode, the target-candidate difficulty similarity score is defined as:

$$S_{\text{target}_{diff}}(q_{u,|\mathbf{s}_u|+1}, q_{u,\tau}) = 1 - \left|d(q_{u,|\mathbf{s}_u|+1}) - d(q_{u,\tau})\right|, \tag{4}$$

where $q_{u,\tau}$ denotes the question following the candidate episode.

Recency Score. The recency score is defined as

$$S_{\text{rec}}(\tau) = \frac{\tau - 1}{|\mathbf{s}_u|}, \tag{5}$$

where τ denotes the temporal index of the candidate episode and $|\mathbf{s}_u|$ represents the length of the target student's learning trajectory. The score increases as the candidate episode becomes closer to the target interaction.

The final self-evidence retrieval score is computed as

$$S_{\text{self}} = \lambda_h S_{\text{traj}} + \lambda_q S_{\text{target}_{\text{diff}}} + \lambda_t S_{\text{rec}}, \tag{6}$$

where λ_h, λ_q, and λ_t are non-negative weighting coefficients satisfying $\lambda_h + \lambda_q + \lambda_t = 1$.

In our experiments, we set λ_h, λ_q, and λ_t to 0.65, 0.30, and 0.05, respectively.

To prevent homogeneous demonstrations, if all retrieved demonstrations have the same follow-up response label, we include the highest-ranked episode with the opposite label as counterevidence.

Details of Peer-Evidence Retrieval. For selecting episodes from peers, we rank the top-K_{peer} candidate episodes separately from the candidates followed by correct and incorrect responses. We then combine the two sets, thereby providing outcome-contrastive peer evidence by separately retrieving peers with correct and incorrect outcomes.

Implementation Details. In the main experimental setting, we set the episode length ω to 5. For self-view retrieval, we retrieve three episodes for DBE-KT22 and NIPS34 and one episode for XES3G5M, with counterevidence added when retrieved episodes show label homogeneity. For peer-view retrieval, we select two correct and two incorrect episodes to provide balanced demonstrations.

Prompt Templates for STEER-KT. Fig 2 illustrates the STEER-KT prompt template. The prompt presents the target episode twice: once with textual metadata and once as a compact statistics-only pattern summary. For in-context demonstrations, we also provide the empirical question correct rate as a question-level statistical signal.

D Ablation Analysis Details.

No-Retrieval. We remove both the self- and peer-evidence while retaining all other conditions, including the target episode and all question-level statistical signals.

Thus, this variant differs from full STEER-KT in the absence of retrieved episodes. Full STEER-KT improves Macro-F1 over this control by 2.7, 2.3, and 5.0 percentage points on DBE-KT22, NIPS34, and XES3G5M, respectively. Each single-evidence variant also outperforms the no-retrieval control, while their combination achieves the highest Macro-F1 across all three datasets.

Removing Question-Level Difficulty Signals. To investigate the contribution of question-level difficulty signals, including the difficulty $d(\cdot)$ and the difficulty label $\ell(\cdot)$, we conduct three ablation experiments by removing difficulty information from different stages of the framework. Specifically, we remove difficulty information from (1) the retrieved demonstrations while retaining difficulty-aware retrieval (w/o Difficulty in Prompt), (2) both self-view and peer-view retrieval while retaining difficulty information in the demonstrations (w/o Difficulty in Retrieval), and (3) both retrieval and prompting stages (w/o Difficulty in Retrieval & Prompt). All other components of the framework remain unchanged. The results are shown in Table 3. These experiments allow us to distinguish the contributions of difficulty-aware evidence selection and difficulty-informed in-context demonstrations. We additionally report a statistical-only baseline that predicts responses using question-level statistics with a fixed 0.5 threshold, without using an LLM. This serves to assess whether the performance gains primarily arise from the statistical signals themselves or from their integration into the LLM-based framework.

Full-Learning Trajectory Peer Matching. To isolate the effect of retrieval granularity, we replace the localized episode representation used for peer matching (Section 3.1) with each student's full learning trajectory available before their target-question attempt. The candidate pool consists of other students who attempted the target question. After ranking candidates using full-learning trajectory similarity, we provide the same length-ω episode immediately preceding each selected peer's target-question attempt. The number of peer demonstrations, outcome balance, target context, and prompt format are identical to STEER-KT. Thus, this variant changes only the history representation used for peer ranking while keeping the evidence presented to the LLM fixed.

This variant decreases Macro-F1 by 2.3, 3.9, and 1.7 percentage points on DBE-KT22, NIPS34, and XES3G5M, respectively. These results suggest that retrieving learning episodes provides more relevant signals than global student histories for identifying peer episodes useful for target prediction.

Random Retrieval. We replace the difficulty-aware episode matching (Section 3.3) strategy with random episode selection. Specifically, self- and peer-evidence are randomly sampled from the same candidate eligibility criteria as STEER-KT. All other components, including the evidence representation, prompt template, and prediction procedure, remain unchanged. The results are shown in Table 3. This comparison helps distinguish the benefit of providing episode-level demonstrations from the additional gains obtained through informed episode matching.

E Generalizability across LLM Backbones.

We use GPT-5.2 for the main evaluation and further evaluate our approach with Qwen3-8B and Llama-3.1-8B-Instruct. Table 5 shows that STEER-KT consistently outperforms existing methods on DBE-KT22 and NIPS34 across different LLM backbones, while achieving competitive performance on XES3G5M. The overall performance varies depending on the backbone.

F Hyperparameter Sensitivity.

We investigate the sensitivity of STEER-KT to three retrieval hyperparameters: the episode window size, the number of retrieved peer episodes (K_{peer}), and the number of retrieved self episodes (K_{self}). In Table 6, the results show that the optimal configurations vary across datasets. A window size of 5 performs best on DBE-KT22 and XES3G5M, whereas a larger window of 10 yields the best results on NIPS34. Similarly, the optimal numbers of peer and self episodes differ across datasets, suggesting that the effectiveness of retrieval configurations depends on dataset characteristics. Nevertheless, the default configuration ($window = 5$, $K_{peer} = 2$, and $K_{self} = 3$) achieves strong overall performance across all three datasets, indicating that STEER-KT does not rely on a single dataset-specific configuration to achieve effective performance.

Table 5: Performance of STEER-KT across different LLM backbones on three public datasets. Best and second-best results are highlighted in bold and underlined, respectively.

LLM Backbone	Model	DBE-KT22		NIPS34		XES3G5M	
		Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
GPT 5.2	EFKT [7]	0.550 ± 0.008	0.559 ± 0.007	0.599 ± 0.002	0.603 ± 0.019	0.589 ± 0.008	0.685 ± 0.009
	HISE-KT [5]	0.626 ± 0.014	0.640 ± 0.014	0.643 ± 0.005	0.643 ± 0.005	0.718 ± 0.016	0.856 ± 0.008
	RAG-KT [6]	0.612 ± 0.012	0.672 ± 0.004	0.655 ± 0.002	0.656 ± 0.002	0.708 ± 0.016	**0.863 ± 0.006**
	STEER-KT (Ours)	**0.692 ± 0.004**	**0.713 ± 0.004**	**0.662 ± 0.003**	**0.662 ± 0.003**	**0.738 ± 0.004**	0.854 ± 0.002
Llama-3.1-8B-Instruct	EFKT [7]	0.464 ± 0.029	0.536 ± 0.031	0.490 ± 0.015	0.518 ± 0.022	0.498 ± 0.037	0.674 ± 0.017
	HISE-KT [5]	0.393 ± 0.014	0.584 ± 0.003	0.440 ± 0.040	0.531 ± 0.006	0.501 ± 0.011	**0.817 ± 0.001**
	RAG-KT [6]	0.583 ± 0.023	0.586 ± 0.028	0.578 ± 0.008	0.585 ± 0.004	0.661 ± 0.034	0.792 ± 0.056
	STEER-KT (Ours)	**0.608 ± 0.007**	**0.640 ± 0.011**	**0.630 ± 0.007**	**0.631 ± 0.006**	**0.663 ± 0.010**	0.815 ± 0.017
Qwen3-8B	EFKT [7]	0.552 ± 0.023	0.553 ± 0.021	0.564 ± 0.011	0.579 ± 0.008	0.558 ± 0.010	0.637 ± 0.008
	HISE-KT [5]	0.618 ± 0.003	0.618 ± 0.003	0.567 ± 0.006	0.596 ± 0.003	0.676 ± 0.012	0.757 ± 0.013
	RAG-KT [6]	0.535 ± 0.008	0.542 ± 0.007	0.557 ± 0.006	0.596 ± 0.003	**0.715 ± 0.002**	**0.796 ± 0.004**
	STEER-KT (Ours)	**0.632 ± 0.007**	**0.633 ± 0.007**	**0.608 ± 0.003**	**0.623 ± 0.001**	0.686 ± 0.003	0.779 ± 0.004

Table 6: Hyperparameter sensitivity analysis of STEER-KT on three public datasets.

	Value	DBE-KT22		NIPS34		XES3G5M	
		Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
window ($K_{peer}=2, K_{self}=3$)	3	0.619 ± 0.007	0.647 ± 0.007	0.658 ± 0.005	0.658 ± 0.005	0.737 ± 0.004	0.852 ± 0.002
	5	0.692 ± 0.004	0.713 ± 0.004	0.662 ± 0.003	0.662 ± 0.003	0.738 ± 0.004	0.854 ± 0.002
	10	0.651 ± 0.005	0.679 ± 0.005	0.671 ± 0.006	0.671 ± 0.006	0.724 ± 0.003	0.847 ± 0.001
K_{peer} ($window=5, K_{self}=3$)	1	0.698 ± 0.005	0.719 ± 0.004	0.662 ± 0.002	0.662 ± 0.003	0.729 ± 0.004	0.847 ± 0.002
	2	0.692 ± 0.004	0.713 ± 0.004	0.662 ± 0.002	0.662 ± 0.003	0.738 ± 0.004	0.854 ± 0.002
	3	0.670 ± 0.005	0.694 ± 0.004	0.664 ± 0.003	0.664 ± 0.003	0.737 ± 0.005	0.851 ± 0.002
K_{self} ($window=5, K_{peer}=2$)	1	0.671 ± 0.001	0.689 ± 0.000	0.663 ± 0.001	0.663 ± 0.001	0.738 ± 0.005	0.854 ± 0.002
	2	0.685 ± 0.005	0.706 ± 0.004	0.669 ± 0.004	0.669 ± 0.004	0.732 ± 0.004	0.847 ± 0.003
	3	0.692 ± 0.004	0.713 ± 0.004	0.662 ± 0.003	0.662 ± 0.003	0.738 ± 0.004	0.854 ± 0.002

G Related Work

Traditional Deep Learning based Knowledge Tracing. Traditional deep learning-based knowledge tracing (KT) models encode students' historical learning interactions into latent representations for predicting future performance. DKT [12] pioneered the use of recurrent neural networks for KT by learning latent representations of students' knowledge directly from interaction sequences. SAKT [11] introduced a self-attention mechanism to selectively attend to relevant historical interactions, enabling more effective modeling of long-range dependencies. SimpleKT [8] proposed a Transformer-based model with a simplified architecture, demonstrating that effective sequence modeling can achieve strong performance with minimal architectural complexity.

Training-free LLM-based Knowledge Tracing. Large language models (LLMs) have shown promising potential for KT by leveraging question-response interactions provided as in-context demonstrations. EFKT [7] introduces training-free LLM-based knowledge tracing by leveraging question-response interactions as in-context examples for few-shot prediction. HISE-KT [5] constructs a heterogeneous information network over students, questions, concepts, and difficulty levels to retrieve similar students through meta-path-based matching. It provides the retrieved students' interaction histories corresponding to the target concept as contextual evidence for LLM-based prediction. RAG-KT [6] was originally designed for cross-platform KT by using question-group abstractions to organize heterogeneous educational data into a shared retrieval space. It retrieves similar students using a fusion of behavioral, structural, and ability similarities, aggregates their performance statistics across multiple educational dimensions, and provides the resulting structured context to the LLM for prediction. In our setting, we adapt RAG-KT to in-domain retrieval by constructing and querying its knowledge base using only the training data from each dataset.

References

[1] Ghodai Abdelrahman, Sherif Abdelfattah, Qing Wang, and Yu Lin. 2022. DBE-KT22: A knowledge tracing dataset based on online student evaluation. *arXiv preprint arXiv:2208.12651* (2022).

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda

Askell, et al. 2020. Language models are few-shot learners. *NeurIPS* (2020).
[3] Youngduck Choi, Youngnam Lee, Junghyun Cho, Jineon Baek, Byungsoo Kim, Yeongmin Cha, Dongmin Shin, Chan Bae, and Jaewe Heo. 2020. Towards an appropriate query, key, and value computation for knowledge tracing. In *Proceedings of the seventh ACM conference on learning@ scale*. 341–344.
[4] Albert T Corbett and John R Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction* 4, 4 (1994), 253–278.
[5] Zhiyi Duan, Zixing Shi, Hongyu Yuan, and Qi Wang. 2026. HISE-KT: Synergizing heterogeneous information networks and llms for explainable knowledge tracing with meta-path optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 14693–14701.
[6] Zhiyi Duan, Hongyu Yuan, and Rui Liu. 2026. RAG-KT: Cross-platform Explainable Knowledge Tracing with Multi-view Fusion Retrieval Generation. In *Findings of the Association for Computational Linguistics: ACL 2026*. 5217–5232.
[7] Haoxuan Li, Jifan Yu, Yuanxin Ouyang, Zhuang Liu, Wenge Rong, Huiqin Liu, Juanzi Li, and Zhang Xiong. 2025. Explainable few-shot knowledge tracing. *Frontiers of Digital Education* 2, 4 (2025), 34.
[8] Zitao Liu, Qiongqiong Liu, Jiahao Chen, Shuyan Huang, and Weiqi Luo. 2023. SimpleKT: a simple but tough-to-beat baseline for knowledge tracing. *arXiv preprint arXiv:2302.06881* (2023).
[9] Zitao Liu, Qiongqiong Liu, Jiahao Chen, Shuyan Huang, Jiliang Tang, and Weiqi Luo. 2022. pyKT: a python library to benchmark deep learning based knowledge tracing models. *Advances in Neural Information Processing Systems* 35 (2022), 18542–18555.
[10] Zitao Liu, Qiongqiong Liu, Teng Guo, Jiahao Chen, Shuyan Huang, Xiangyu Zhao, Jiliang Tang, Weiqi Luo, and Jian Weng. 2023. XES3G5M: A knowledge tracing benchmark dataset with auxiliary information. *Advances in Neural Information Processing Systems* 36 (2023), 32958–32970.
[11] Shalini Pandey and George Karypis. 2019. A self-attentive model for knowledge tracing. *arXiv preprint arXiv:1907.06837* (2019).
[12] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. 2015. Deep knowledge tracing. *Advances in neural information processing systems* 28 (2015).
[13] Zichao Wang, Angus Lamb, Evgeny Saveliev, Pashmina Cameron, Yordan Zaykov, José Miguel Hernández-Lobato, Richard E Turner, Richard G Baraniuk, Craig Barton, Simon Peyton Jones, et al. 2020. Instructions and guide for diagnostic questions: The neurips 2020 education challenge. *arXiv preprint arXiv:2007.12061* (2020).
[14] Chun Kit Yeung and Dit Yan Yeung. 2018. Addressing two problems in deep knowledge tracing via prediction-consistent regularization. In *Proceedings of the 5th ACM Conference on Learning @ Scale*. ACM, 5:1–5:10.
[15] Jiani Zhang, Xingjian Shi, Irwin King, and Dit-Yan Yeung. 2017. Dynamic key-value memory networks for knowledge tracing. In *Proceedings of the 26th international conference on World Wide Web*. 765–774.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009